



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТАЭ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ 8
по дисциплине «Автоматизированные системы управления элементами
промышленных и административных объектов»

АСУ П по поддержанию температуры в оранжерее

Студент группы *ИКБО-50-23, Астахов С.П.*

(подпись)

Преподаватель *Щербаков К.В.*

(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

1. Выбор реляционной СУБД

1.1. Поиск реляционных СУБД

Для проектируемой автоматизированной системы управления (АСУ) поддержания температуры в оранжерее необходимо выбрать реляционную СУБД, которая обеспечит:

- хранение показаний датчиков температуры, влажности, CO₂;
- учёт работы исполнительных устройств (обогреватели, вентиляция, увлажнители);
- логирование аварийных событий;
- управление зонами оранжереи и растениями

Рассмотрены три СУБД: PostgreSQL, MySQL (Community Edition), SQLite

1.1.1. PostgreSQL

Общая характеристика: Свободная объектно-реляционная СУБД, ориентированная на соответствие стандарту SQL, надёжность и расширяемость.

Возможности, релевантные для проектируемой АСУ:

- Полная поддержка ACID-транзакций (журнал команд и событий).
- Тип JSONB для гибкого хранения параметров управления (например, {"режим": "ночной", "мощность": 80}).
- Механизм MVCC для обработки одновременных записей с нескольких датчиков.
- Хранимые процедуры для автоматического расчёта необходимого тепла.
- Расширение TimescaleDB для эффективной работы с временными рядами (показания датчиков).

1.1.2. MySQL (Community Edition)

Общая характеристика: Самая популярная СУБД с открытым исходным кодом для веб-приложений. Отличается высокой скоростью операций чтения

и простотой администрирования.

Возможности, релевантные для проектируемой АСУ:

- Движок InnoDB с поддержкой транзакций и внешних ключей.
- Тип JSON (с версии 5.7) для хранения параметров.
- Простота настройки репликации.

1.1.3. SQLite

Общая характеристика: Встраиваемая библиотека, вся БД – в одном файле.

Возможности, релевантные для проектируемой АСУ:

- Нулевое администрирование – идеально для прототипа.
- Поддержка ACID, большинства конструкций SQL.

1.2. Сравнительный анализ

Для выбора оптимальной СУБД выделим ключевые критерии, продиктованные функциональными и нефункциональными требованиями к АСУ «Семантический подборщик».

Критерий	PostgreSQL	MySQL	SQLite
Поддержка конкурентного доступа	Отличная (MVCC)	Хорошая (InnoDB)	Отсутствует (только один писатель одновременно)
Работа с временными рядами	Отличная(через TimescaleDB)	Средняя	Плохая
Работа с JSON	Отличная(JSONB)	Хорошая	Базовая
Сложность администрирования	Высокая	Средняя	Нулевая
Соответствие стандарту SQL	Очень высокое	Среднее	Высокое

1.3. Преимущества и недостатки рассмотренных СУБД

1. PostgreSQL

○ Преимущества:

- Гибкость хранения конфигураций: Тип JSONB позволяет хранить гибкие настройки устройств управления (например, {"мощность": 80, "режим": "ночной", "таймер": 3600}) и параметры аварийных сценариев в едином поле без потери

возможности индексации и быстрого поиска.

- Транзакционная целостность: Критически важна для процесса автоматического управления климатом, где при аварийном отключении питания или сбое датчика недопустимы частичные изменения в логах команд и событиях системы.
- Расширяемость: В будущем позволит добавить в БД расширение TimescaleDB для эффективного хранения и анализа временных рядов (показания датчиков температуры и влажности), что критически важно для долгосрочного мониторинга микроклимата.

○ Недостатки:

- Требуется более тщательной первоначальной настройки параметров использования памяти и дискового ввода-вывода.
- Стандартная утилита `pg_dump` создаёт резервные копии несколько медленнее, чем специализированные средства MySQL.

2. MySQL

○ Преимущества:

- Высокая производительность на простых операциях чтения текущих показаний датчиков по идентификатору зоны, что важно для панели оперативного управления (dashboard) агронома.
- Огромное количество готовых решений и документации по интеграции с SCADA-системами и платформами IoT.

○ Недостатки:

- Вольная трактовка стандарта SQL (например, режим `ONLY_FULL_GROUP_BY` по умолчанию был отключён в старых версиях), что может привести к скрытым логическим ошибкам в аналитических отчётах по эффективности работы устройств управления.

- Работа с JSON менее производительна и удобна, чем с JSONB в PostgreSQL, особенно при глубокой вложенности параметров сценариев автоматизации.

3. SQLite

○ Преимущества:

- Идеальная среда для быстрой проверки SQL-скриптов создания таблиц и тестового наполнения данными на локальной машине разработчика без развёртывания сервера. Подходит для прототипирования логики управления на Raspberry Pi или Arduino.

○ Недостатки:

- Невозможность использования в промышленной АСУ оранжереи: Отсутствие сетевого доступа и блокировка всей базы на время записи делают SQLite непригодным для системы, где несколько контроллеров одновременно записывают показания с десятков датчиков.
- Ограниченная поддержка команды ALTER TABLE (например, для удаления столбца требуется пересоздание таблицы), что затруднит итеративное развитие схемы БД при добавлении новых типов датчиков (например, pH почвы, освещённость, CO₂).

1.4. Выбор СУБД для создания базы данных проектируемой АСУ

На основании проведённого анализа в качестве СУБД для АСУ «Семантический подборщик» выбрана PostgreSQL.

Обоснование выбора:

1. Соответствие требованиям предметной области: ER-диаграмма содержит сущности «показания_датчиков» и «события_системы», которые генерируются непрерывно. PostgreSQL с расширением TimescaleDB обеспечивает оптимальное хранение и быструю выборку

временных рядов.

2. Надёжность учёта операций: Процесс автоматического включения/выключения обогревателей и вентиляции требует строгой транзакционности, чтобы при сбое не произошло рассогласования между реальным состоянием устройства и записью в лог. PostgreSQL обеспечивает этот уровень надёжности.
3. Гибкость конфигурирования: Различные сценарии управления микроклиматом (дневной/ночной режим, защита от заморозков, вентиляция при превышении влажности) удобно хранить в JSONB-полях таблицы «устройства_управления».
4. Перспектива развития: Внедрение прогнозной аналитики на основе исторических данных (предсказание заморозков, оптимизация энергопотребления) в будущем легко реализуется с помощью расширения pgvector для хранения эмбедингов или TimescaleDB для прогнозов временных рядов.

2. Создание базы данных в выбранной СУБД и заполнение ее данными

Листинг 1: Создание БД

```
-- АСУ ПОДДЕРЖАНИЯ ТЕМПЕРАТУРЫ В ОРАНЖЕРЕЕ
-- Скрипт создания базы данных и таблиц
CREATE DATABASE greenhouse_asu;

USE greenhouse_asu;

-- Таблица: зоны оранжереи
CREATE TABLE zones (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    area DECIMAL(10,2),
    target_temp_min DECIMAL(5,2) NOT NULL,
    target_temp_max DECIMAL(5,2) NOT NULL,
    target_humidity_min INT,
    target_humidity_max INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица: растения
CREATE TABLE plants (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    variety VARCHAR(100),
    zone_id INT,
    optimal_day_temp_min DECIMAL(5,2),
    optimal_day_temp_max DECIMAL(5,2),
    optimal_night_temp_min DECIMAL(5,2),
    optimal_night_temp_max DECIMAL(5,2),
    allowed_range_min DECIMAL(5,2),
    allowed_range_max DECIMAL(5,2),
    planting_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

-- Таблица: датчики
CREATE TABLE sensors (
    id INT PRIMARY KEY AUTO_INCREMENT,
    serial_number VARCHAR(50) UNIQUE NOT NULL,
    type VARCHAR(50) NOT NULL,
    zone_id INT,
    location VARCHAR(255),
    status VARCHAR(50) DEFAULT 'active',
    install_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_calibration DATE,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET
NULL
);

-- Таблица: показания датчиков
CREATE TABLE sensor_readings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    sensor_id INT NOT NULL,
```

```

value DECIMAL(10,2) NOT NULL,
unit VARCHAR(20),
reading_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
is_alarm BOOLEAN DEFAULT FALSE,
FOREIGN KEY (sensor_id) REFERENCES sensors(id) ON
DELETE CASCADE
);

-- Таблица: устройства управления
CREATE TABLE control_devices (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    type VARCHAR(50) NOT NULL,
    zone_id INT,
    power DECIMAL(10,2),
    state VARCHAR(50) DEFAULT 'off',
    control_mode VARCHAR(50) DEFAULT 'auto',
    install_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_turned_on TIMESTAMP,
    last_turned_off TIMESTAMP,
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE
SET NULL
);

-- Таблица: команды управления
CREATE TABLE control_commands (
    id INT PRIMARY KEY AUTO_INCREMENT,
    device_id INT NOT NULL,
    command VARCHAR(50) NOT NULL,
    parameters JSON,
    reason VARCHAR(255),
    initiator VARCHAR(100) DEFAULT 'ACS',
    command_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    execution_status VARCHAR(50) DEFAULT 'sent',
    FOREIGN KEY (device_id) REFERENCES
control_devices(id) ON DELETE CASCADE
);

-- Таблица: события системы
CREATE TABLE system_events (
    id INT PRIMARY KEY AUTO_INCREMENT,
    type VARCHAR(50) NOT NULL,
    zone_id INT,
    sensor_id INT,
    message TEXT NOT NULL,
    deviation_value DECIMAL(10,2),
    threshold_value DECIMAL(10,2),
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_time TIMESTAMP,
    status VARCHAR(50) DEFAULT 'active',
    FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE
SET NULL,
FOREIGN KEY (sensor_id) REFERENCES sensors(id) ON
DELETE SET NULL
);

-- Таблица: пользователи
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,

```



```

password_hash VARCHAR(255) NOT NULL,
role VARCHAR(50) DEFAULT 'agronomist',
zone_id INT,
phone VARCHAR(20),
is_active BOOLEAN DEFAULT TRUE,
last_login TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (zone_id) REFERENCES zones(id) ON DELETE SET NULL
);

```

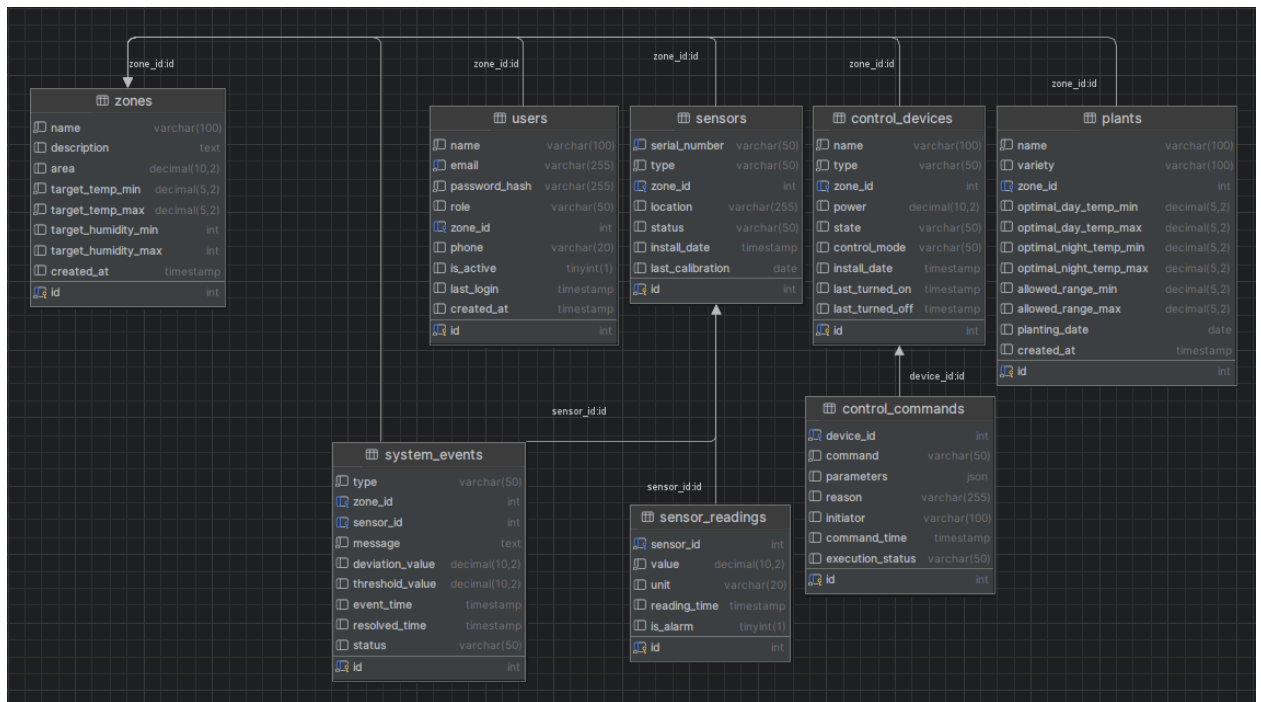


Рисунок 1 - Созданная БД

Листинг 2: Заполнение БД

```

-- Заполнение таблиц

-- Зоны оранжереи (5 зон)
INSERT INTO zones (name, description, area, target_temp_min, target_temp_max,
target_humidity_min, target_humidity_max) VALUES
('Sector A - Tomatoes', 'Indeterminate tomatoes', 120.5, 20.0, 26.0, 60, 75),
('Sector B - Cucumbers', 'Parthenocarpic cucumbers', 95.0, 22.0, 28.0, 70, 85),
('Sector C - Seedlings', 'Vegetable seedlings', 45.0, 18.0, 22.0, 50, 65),
('Sector D - Flowers', 'Roses and chrysanthemums', 80.0, 16.0, 24.0, 55, 70),
('Sector E - Sweet Pepper', 'Bell pepper', 60.0, 21.0, 27.0, 65, 80);

-- Растения
INSERT INTO plants (name, variety, zone_id, optimal_day_temp_min,
optimal_day_temp_max, optimal_night_temp_min, optimal_night_temp_max,
allowed_range_min, allowed_range_max, planting_date) VALUES
('Tomato', 'Bull Heart', 1, 22.0, 26.0, 16.0, 18.0, 12.0, 32.0, '2026-03-15'),
('Tomato', 'Red Cherry', 1, 20.0, 25.0, 15.0, 17.0, 10.0, 30.0, '2026-03-20'),
('Cucumber', 'Courage F1', 2, 24.0, 28.0, 18.0, 20.0, 14.0, 32.0, '2026-03-10'),
('Cucumber', 'German F1', 2, 23.0, 27.0, 17.0, 19.0, 12.0, 32.0, '2026-03-12'),
('Sweet Pepper', 'California Wonder', 5, 23.0, 27.0, 17.0, 19.0, 12.0, 30.0, '2026-02-01'),
('Rose', 'Etoile de Hollande', 4, 18.0, 22.0, 14.0, 16.0, 8.0, 28.0, '2026-01-15'),

```

```

('Basil', 'Greek', 3, 20.0, 24.0, 15.0, 17.0, 10.0, 28.0, '2026-04-01'),
('Lettuce', 'Iceberg', 3, 15.0, 20.0, 10.0, 14.0, 5.0, 25.0, '2026-04-05'),
('Eggplant', 'Black Beauty', 5, 22.0, 28.0, 18.0, 20.0, 12.0, 32.0, '2026-03-01'),
('Strawberry', 'Albion', 3, 18.0, 24.0, 10.0, 14.0, 5.0, 28.0, '2026-02-10');

-- Датчики
INSERT INTO sensors (serial_number, type, zone_id, location, status,
last_calibration) VALUES
('TMP-001', 'temperature', 1, 'Center of Sector A, height 1.5m', 'active', '2026-01-15'),
('TMP-002', 'temperature', 1, 'Northern part of Sector A', 'active', '2026-01-15'),
('TMP-003', 'temperature', 2, 'Center of Sector B', 'active', '2026-01-20'),
('TMP-004', 'temperature', 3, 'Shelf 1', 'active', '2026-02-01'),
('TMP-005', 'temperature', 4, 'Center of Sector D', 'active', '2026-01-10'),
('TMP-006', 'temperature', 5, 'Center of Sector E', 'maintenance', '2025-12-01'),
('HUM-001', 'humidity', 1, 'Center of Sector A', 'active', '2026-01-15'),
('HUM-002', 'humidity', 2, 'Southern part of Sector B', 'active', '2026-01-20'),
('HUM-003', 'humidity', 3, 'Shelf 2', 'active', '2026-02-01'),
('CO2-001', 'co2', 1, 'Center of Sector A', 'active', '2026-01-15');

-- Показания датчиков
INSERT INTO sensor_readings (sensor_id, value, unit, reading_time, is_alarm) VALUES
(1, 23.5, '°C', '2026-05-16 08:00:00', FALSE),
(1, 24.2, '°C', '2026-05-16 12:00:00', FALSE),
(1, 26.8, '°C', '2026-05-16 15:30:00', FALSE),
(1, 27.5, '°C', '2026-05-16 16:00:00', TRUE),
(2, 22.1, '°C', '2026-05-16 08:00:00', FALSE),
(2, 23.4, '°C', '2026-05-16 14:00:00', FALSE),
(3, 25.3, '°C', '2026-05-16 08:00:00', FALSE),
(3, 29.1, '°C', '2026-05-16 13:00:00', TRUE),
(4, 19.8, '°C', '2026-05-16 08:00:00', FALSE),
(4, 21.2, '°C', '2026-05-16 11:00:00', FALSE),
(5, 17.5, '°C', '2026-05-16 06:00:00', FALSE),
(5, 23.0, '°C', '2026-05-16 14:00:00', FALSE),
(7, 68.0, '%', '2026-05-16 08:00:00', FALSE),
(7, 82.0, '%', '2026-05-16 16:00:00', TRUE),
(10, 420.0, 'ppm', '2026-05-16 08:00:00', FALSE);

-- Устройства управления
INSERT INTO control_devices (name, type, zone_id, power, state, control_mode,
last_turned_on, last_turned_off) VALUES
('Gas Heater No.1', 'heater', 1, 5.5, 'off', 'auto', '2026-05-15 22:00:00', '2026-05-16 06:00:00'),
('Gas Heater No.2', 'heater', 2, 4.8, 'off', 'auto', '2026-05-15 21:30:00', '2026-05-16 05:30:00'),
('Exhaust Fan No.1', 'fan', 1, 0.75, 'on', 'auto', '2026-05-16 10:00:00', NULL),
('Exhaust Fan No.2', 'fan', 2, 0.75, 'off', 'auto', '2026-05-15 14:00:00', '2026-05-15 15:00:00'),
('Ultrasonic Humidifier No.1', 'humidifier', 1, 0.3, 'off', 'auto', '2026-05-16 06:00:00', '2026-05-16 08:00:00'),
('Ultrasonic Humidifier No.2', 'humidifier', 2, 0.3, 'on', 'auto', '2026-05-16 07:00:00', NULL),
('Shading System', 'shading', 1, 0.2, 'off', 'manual', '2026-05-15 12:00:00', '2026-05-15 16:00:00'),
('Circulation Fan', 'fan', 3, 0.4, 'on', 'auto', '2026-05-16 08:00:00', NULL);

-- Команды управления
INSERT INTO control_commands (device_id, command, parameters, reason, initiator,
execution_status) VALUES
(1, 'ON', '{"power": 80}', 'temperature below target by 2.5°C', 'ACS', 'executed'),
(1, 'OFF', '{}', 'temperature reached target', 'ACS', 'executed'),

```

```

(2, 'ON', '{"power": 75}', 'night temperature drop', 'ACS', 'executed'),
(3, 'ON', '{"speed": 3}', 'humidity exceeded 75%', 'ACS', 'executed'),
(4, 'ON', '{"speed": 2}', 'CO2 level exceeded 800 ppm', 'ACS', 'executed'),
(4, 'OFF', '{}', 'CO2 normalized', 'ACS', 'executed'),
(5, 'ON', '{"intensity": 60}', 'humidity fell below 60%', 'ACS', 'executed'),
(5, 'OFF', '{}', 'humidity normalized', 'ACS', 'executed'),
(6, 'ON', '{"intensity": 70}', 'humidity fell below 65%', 'ACS', 'executed'),
(7, 'ON', '{"level": 80}', 'temperature exceeded 28°C', 'agronomist', 'executed');

-- События системы
INSERT INTO system_events (type, zone_id, sensor_id, message, deviation_value,
threshold_value, status, resolved_time) VALUES
('ALARM', 1, 1, 'Critical temperature exceedance: 34.2°C at threshold 30°C', 34.2,
30.0, 'resolved', '2026-05-16 10:30:00'),
('ALARM', 2, 3, 'Critical temperature exceedance: 33.1°C at threshold 32°C', 33.1,
32.0, 'resolved', '2026-05-16 14:00:00'),
('WARNING', 1, 7, 'High humidity: 85% at threshold 75%', 85.0, 75.0, 'active', NULL),
('WARNING', 2, 8, 'Low humidity: 45% at threshold 55%', 45.0, 55.0, 'active', NULL),
('WARNING', 3, 4, 'Low temperature: 8°C at threshold 10°C', 8.0, 10.0, 'resolved',
'2026-05-16 09:00:00'),
('INFO', 1, NULL, 'Planned ventilation shutdown for maintenance', NULL, NULL,
'resolved', '2026-05-15 18:00:00'),
('ALARM', 4, 5, 'Heater failure detected', NULL, NULL, 'resolved', '2026-05-16
11:00:00'),
('WARNING', 5, 6, 'Sensor calibration required', NULL, NULL, 'active', NULL),
('INFO', 2, NULL, 'Daily backup completed', NULL, NULL, 'resolved', '2026-05-16
02:00:00'),
('WARNING', 1, 2, 'Temperature sensor reading unstable', NULL, NULL, 'active', NULL);

-- Пользователи
INSERT INTO users (name, email, password_hash, role, zone_id, phone, is_active)
VALUES
('Alexey Agronomov', 'agronomist@greenhouse.ru', 'hash_agro_123', 'agronomist', 1,
'+79161234567', TRUE),
('Maria Engineerova', 'engineer@greenhouse.ru', 'hash_eng_456', 'engineer', NULL,
'+79261234568', TRUE),
('Ivan Administratorov', 'admin@greenhouse.ru', 'hash_admin_789', 'admin', NULL,
'+79361234569', TRUE),
('Olga Observer', 'observer@greenhouse.ru', 'hash_obs_321', 'observer', 2,
'+79461234570', TRUE),
('Petr Petrov', 'petrov@greenhouse.ru', 'hash_petr_111', 'agronomist', 3,
'+79561234571', TRUE),
('Svetlana Sidorova', 'sveta@greenhouse.ru', 'hash_sveta_222', 'engineer', NULL,
'+79661234572', FALSE),
('Dmitry Dmitriev', 'dmitry@greenhouse.ru', 'hash_dmitry_333', 'agronomist', 5,
'+79761234573', TRUE),
('Elena Elenova', 'elena@greenhouse.ru', 'hash_elena_444', 'observer', 4,
'+79861234574', TRUE);

```

id	device_id	command	parameters	reason
1	1	1 ON	{"power": 80}	temperature below target by 2.5°C
2	2	1 OFF	{}	temperature reached target
3	3	2 ON	{"power": 75}	night temperature drop
4	4	3 ON	{"speed": 3}	humidity exceeded 75%
5	5	4 ON	{"speed": 2}	CO2 level exceeded 800 ppm
6	6	4 OFF	{}	CO2 normalized
7	7	5 ON	{"intensity": 60}	humidity fell below 60%
8	8	5 OFF	{}	humidity normalized
9	9	6 ON	{"intensity": 70}	humidity fell below 65%
10	10	7 ON	{"level": 80}	temperature exceeded 28°C

Рисунок 2 - Заполненная таблица «пользователи»

Листинг 3: Запрос

```
-- Поиск запрос: найти все аварийные события (ALARM) в зоне №1 (Tomatoes)
-- с температурными отклонениями за последние 7 дней

SELECT
    se.id AS event_id,
    se.type AS event_type,
    z.name AS zone_name,
    s.serial_number AS sensor_serial,
    se.message,
    se.deviation_value,
    se.threshold_value,
    se.event_time,
    se.status
FROM system_events se
    JOIN zones z ON se.zone_id = z.id
    LEFT JOIN sensors s ON se.sensor_id = s.id
WHERE se.type = 'ALARM'
    AND se.zone_id = 1
    AND se.event_time >= DATE_SUB(NOW(), INTERVAL 7 DAY)
ORDER BY se.event_time DESC;
```

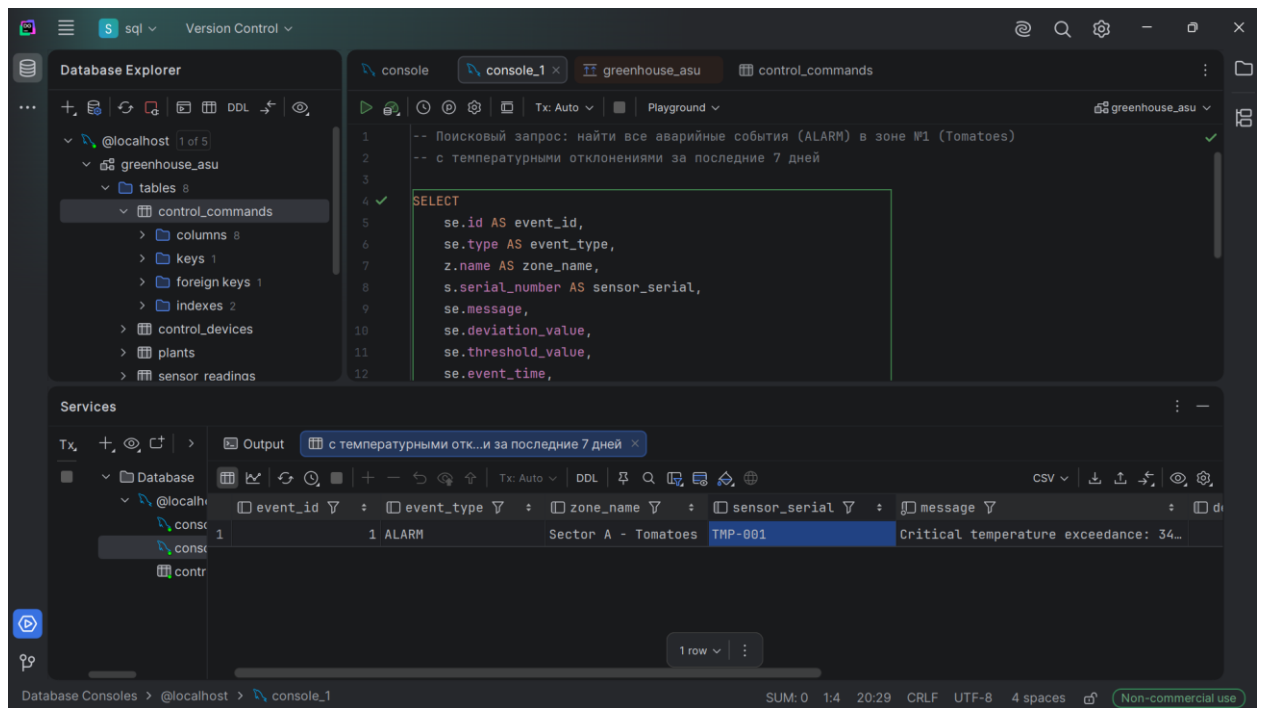


Рисунок 3 - Результат